



05-DARS · 14 TADAN

DOM va hodisalar

DOM daraxti, elementlarni topish va o'zgartirish, hodisalar (events), bubbling va delegatsiya, formalar bilan ishlash, localStorage.

O'rta · ~120 daqiqa

Oldingi darsda JavaScript tilini o'rgandik. Endi uni sahifa bilan bog'laymiz. **DOM (Document Object Model)** — brauzer HTML'dan quradigan va JavaScript orqali boshqarish mumkin bo'lgan **obyektlar daraxti**. Tugma bosilganda oyna ochilishi, forma tekshirilishi, ro'yxatga element qo'shilishi — barchasi DOM bilan ishlash.

DOM daraxti

Quyidagi HTML:

HTML

```
<body>
  <h1 id="title">Vazifalar</h1>
  <ul class="list">
    <li>Non olish</li>
    <li>Dars qilish</li>
  </ul>
</body>
```

brauzerda shunday daraxtga aylanadi:

Matn

```
document
├── html
│   ├── head
│   └── body
│       ├── h1#title
│       │   └── "Vazifalar" (matn tuguni)
│       └── ul.list
│           ├── li
│           │   └── "Non olish"
│           └── li
│               └── "Dars qilish"
```

- Daraxtdagi har bir narsa — **tugun (node)**: element tugunlari, matn tugunlari, izoh tugunlari.
- `document` — daraxtning ildizi va DOM bilan ishlashning kirish nuqtasi.
- Elementlar o'rtasida oilaviy munosabatlar bor: ota (parent), farzand (child), aka-uka (sibling).



DOM ≠ HTML manba kodi

DOM — "jonli" tuzilma. JavaScript uni o'zgartirsa, sahifa darhol yangilanadi, lekin serverdagi HTML fayl o'zgarmaydi. DevTools'dagi **Elements** paneli manba kodni emas, aynan joriy DOM'ni ko'rsatadi.

Elementlarni topish

JavaScript

```
// Zamonaviy va universal usul – CSS selektorlari bilan
const title = document.querySelector('#title');           // birinchi mos
// element yoki null
const firstItem = document.querySelector('.list li');
const items = document.querySelectorAll('.list li');      // NodeList (barcha
// moslar)
items.forEach(li => console.log(li.textContent));
const arr = [...items];                                   // haqiqiy massivga
// aylantirish

// Klassik usullar
document.getElementById('title');
document.getElementsByClassName('list'); // "jonli" HTMLCollection
document.getElementsByTagName('li');

// Element ichidan qidirish
const list = document.querySelector('.list');
list.querySelectorAll('li');

// Daraxt bo'ylab harakatlanish
const li = list.firstElementChild;
li.parentElement;           // ul
li.nextElementSibling;      // ikkinchi li
li.closest('.list');         // eng yaqin mos ajdod (o'zi ham kiradi) – juda
// foydali!
list.children;               // farzand elementlar
```

Elementlarni o'zgartirish

Mazmun

JavaScript

```
const el = document.querySelector('#title');

el.textContent = 'Yangi sarlavha';           // faqat matn – XAVFSIZ
el.innerHTML = 'Bugungi <em>vazifalar</em>'; // HTML sifatida talqin qilinadi
– EHTIYOT BO'LING

// ❌ Hech qachon foydalanuvchi kiritgan ma'lumotni innerHTML ga qo'ymang:
const izoh = '<img src=x onerror="alert(document.cookie)">';
// el.innerHTML = izoh; // XSS hujumi! Skript bajariladi
el.textContent = izoh; // ✅ shunchaki matn sifatida ko'rinadi
```



XSS (Cross-Site Scripting)

`innerHTML` ga tekshirilmagan ma'lumot qo'yish — webdagi eng keng tarqalgan zaifliklardan biri. Hujumchi izoh, ism yoki qidiruv so'zi orqali o'z skriptini boshqa foydalanuvchilar brauzerida bajarishi mumkin. Foydalanuvchi ma'lumoti uchun doim `textContent` dan foydalaning (14-darsda batafsil).

Atributlar va sinflar

JavaScript

```
const img = document.querySelector('img');
img.src = 'yangi.jpg'; // standart atributlar xususiyat
sifatida
img.alt = 'Tavsif';
img.setAttribute('loading', 'lazy');
img.getAttribute('width');
img.removeAttribute('title');
img.hasAttribute('alt');

// data-* atributlar → dataset
// <button data-product-id="42" data-price="15000">
btn.dataset.productId; // '42' (har doim satr!)
Number(btn.dataset.price); // 15000

// Sinflar – uslubni o'zgartirishning to'g'ri usuli
el.classList.add('active');
el.classList.remove('hidden');
el.classList.toggle('open'); // bor bo'lsa olib tashlaydi, yo'q
bo'lsa qo'shadi
el.classList.toggle('dark', isDark); // ikkinchi argument: majburan
qo'shish/olib tashlash
el.classList.contains('active');

// Inline uslublar (kamdan-kam – asosan dinamik qiymatlar uchun)
el.style.width = `${progress}%`;
el.style.setProperty('--accent', '#e11d48'); // CSS o'zgaruvchisi
getComputedStyle(el).fontSize; // haqiqiy hisoblangan uslub
```

Element yaratish, qo'shish va o'chirish

JavaScript

```
const li = document.createElement('li');
li.textContent = 'Yangi vazifa';
li.className = 'task';
li.dataset.id = crypto.randomUUID();

const list = document.querySelector('.list');
list.append(li);           // oxiriga (bir nechta tugun va satr qabul qiladi)
list.prepend(li);         // boshiga
list.before(el); list.after(el); // ro'yxatdan oldin/keyin
li.replaceWith(boshqaEl);
li.remove();               // o'chirish

// HTML satrdan joylash (ishonchli, statik shablon uchun)
list.insertAdjacentHTML('beforeend', '<li class="task">Statik matn</li>');

// <template> – HTML'da tayyor shablon
// <template id="card-tpl"><article class="card"><h3></h3><p></p></article>
// </template>
const tpl = document.querySelector('#card-tpl');
const card = tpl.content.cloneNode(true);
card.querySelector('h3').textContent = 'Sarlavha';
document.body.append(card);
```



Unumdorlik: ko'p elementni bir yo'la qo'shing

Siklda har bir elementni alohida DOM'ga qo'shish har safar Layout'ni qayta hisoblashi mumkin. 1000 ta elementni avval `DocumentFragment` ga yoki massivga yig'ib, keyin bir marta qo'shing:

```
list.append(...elements) yoki list.replaceChildren(...elements) .
```

Hodisalar (Events)

Hodisa — brauzerda sodir bo'lgan narsa: bosish, klavish bosilishi, forma yuborilishi, sahifa yuklanishi, scroll. Biz hodisaga **tinglovchi (listener)** — funksiya biriktiramiz.

```
const btn = document.querySelector('#save');

function saqlash(event) {
  console.log('Hodisa turi:', event.type);           // 'click'
  console.log('Bosilgan element:', event.target);
  console.log('Koordinatalar:', event.clientX, event.clientY);
}

btn.addEventListener('click', saqlash);
btn.removeEventListener('click', saqlash); // olib tashlash uchun AYNI
funksiya kerak

// Variantlar
btn.addEventListener('click', saqlash, { once: true }); // faqat bir marta
window.addEventListener('scroll', onScroll, { passive: true }); // scroll
unumdorligi uchun

// ❌ Eskicha usul (bitta hodisaga faqat bitta ishlovchi):
// btn.onclick = saqlash;
// <button onclick="saqlash()"> - HTML va JS aralashib ketadi
```

Ko'p ishlatiladigan hodisalar

Toifa	Hodisalar
Sichqoncha	click, dblclick, contextmenu, mouseenter / mouseleave, mousemove
Pointer (sichqoncha + sensor)	pointerdown, pointermove, pointerup
Klaviatura	keydown, keyup (event.key : 'Enter', 'Escape', 'a')
Forma	submit, input (har o'zgarishda), change (yakunlanganda), focus, blur, reset
Hujjat/oyina	DOMContentLoaded, load, resize, scroll, beforeunload
Boshqalar	dragstart / drop, copy / paste, animationend, transitionend

Standart xatti-harakatni to'xtatish

JavaScript

```
form.addEventListener('submit', (e) => {
  e.preventDefault();      // sahifa qayta yuklanmasin – ma'lumotni JS orqali
                             yuboramiz
});

link.addEventListener('click', (e) => {
  e.preventDefault();      // havolaga o'tmasin
});

document.addEventListener('keydown', (e) => {
  if ((e.ctrlKey || e.metaKey) && e.key === 's') {
    e.preventDefault();    // brauzerning "Saqlash" oynasi o'rniga o'z
                             funksiyamiz
    saqlash();
  }
});
```

Hodisa tarqalishi: capturing va bubbling

Ichma-ich elementlardan biri bosilganda hodisa uch bosqichdan o'tadi:

1. **Capturing** — `window` dan maqsad elementga qarab pastga tushadi;
2. **Target** — maqsad elementga yetadi;
3. **Bubbling** — maqsad elementdan `window` gacha yuqoriga "ko'tariladi" (pufakchadek).

HTML

```
<div id="karta">
  <button id="tugma">Bos</button>
</div>
<script>
  karta.addEventListener('click', () => console.log('2. karta'));
  tugma.addEventListener('click', () => console.log('1. tugma'));
  document.body.addEventListener('click', () => console.log('3. body'));
  // Tugma bosilsa: 1. tugma → 2. karta → 3. body
</script>
```

- `event.target` — hodisa **haqiqatda** sodir bo'lgan element (bosilgan tugma).
- `event.currentTarget` — tinglovchi **biriktirilgan** element.
- `event.stopPropagation()` — ko'tarilishni to'xtatadi (kamdan-kam va ehtiyotkorlik bilan ishlating).

Hodisa delegatsiyasi (event delegation)

Bubbling tufayli 100 ta tugmaning har biriga tinglovchi qo'yish shart emas — **ularning umumiy otasiga bittasini** qo'yish yetarli. Bu usul dinamik qo'shilgan elementlar uchun ham avtomatik ishlaydi.

JavaScript

```
// ❌ Har bir tugmaga alohida — yangi qo'shilgan elementlar ishlamaydi
document.querySelectorAll('.delete-btn').forEach(btn =>
  btn.addEventListener('click', ...)
);

// ✅ Delegatsiya — bitta tinglovchi
document.querySelector('#todo-list').addEventListener('click', (e) => {
  const btn = e.target.closest('button[data-action]');
  if (!btn) return; // tugmadan boshqa joy
  bosildi
  const item = btn.closest('li');
  const id = item.dataset.id;

  if (btn.dataset.action === 'delete') deleteTodo(id);
  if (btn.dataset.action === 'edit') editTodo(id);
});
```

Formalar bilan ishlash

JavaScript

```
const form = document.querySelector('#signup');

form.addEventListener('submit', (e) => {
  e.preventDefault();

  // 1-usul: alohida maydonlar
  const email = form.elements.email.value.trim();

  // 2-usul: FormData — barcha maydonlarni bir yo'la
  const data = Object.fromEntries(new FormData(form));
  // { email: '...', password: '...', course: 'fullstack' }

  // Bir xil name'li checkbox'lar uchun
  const days = new FormData(form).getAll('days');

  console.log(data, days);
  form.reset();
});

// Real vaqtda kuzatish
form.elements.password.addEventListener('input', (e) => {
  const len = e.target.value.length;
  hint.textContent = len < 8 ? `Yana ${8 - len} ta belgi kerak` : '✅ Yaxshi';
});

// checkbox va select
checkbox.checked;           // true / false
select.value;           // tanlangan option value
select.selectedOptions[0].text;
```

Brauzer xotirasi: localStorage va sessionStorage

JavaScript

```
// Faqat SATR saqlaydi — obyektlar uchun JSON kerak
localStorage.setItem('theme', 'dark');
localStorage.getItem('theme');           // 'dark' (yoki yo'q bo'lsa null)
localStorage.removeItem('theme');
localStorage.clear();

const todos = [{ id: 1, text: 'Dars', done: false }];
localStorage.setItem('todos', JSON.stringify(todos));
const saved = JSON.parse(localStorage.getItem('todos') ?? '[]');

// sessionStorage — xuddi shunday, lekin brauzer yorlig'i yopilganda o'chadi
```

	localStorage	sessionStorage	Cookie
Hajm	~5–10 MB	~5 MB	~4 KB
Muddati	Abadiy (o'chirilguncha)	Yorliq yopilguncha	Belgilangan muddat
Serverga yuboriladimi	Yo'q	Yo'q	Ha, har so'rovda
JS'dan o'qish	Ha	Ha	Ha (HttpOnly bo'lmasa)

 localStorage'ga maxfiy ma'lumot (parol, karta raqami) saqlamang — sahifadagi har qanday skript (shu jumladan XSS orqali kiritilgan) uni o'qiy oladi.

Amaliy misollar

MISOL 1

To'liq Todo ilovasi (CRUD, filtr, localStorage)

Klassik, lekin DOM'ning deyarli barcha tushunchalarini qamrab oladigan loyiha. Arxitektura muhim: **holat (state)** alohida massivda saqlanadi, DOM esa har doim shu holatdan **chiziladi (render)**. Bu React'ning asosiy g'oyasi (12-dars).

```
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vazifalar</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <h1>📝 Vazifalar</h1>
    <form id="todo-form" autocomplete="off">
      <input name="text" placeholder="Yangi vazifa..." required
maxlength="120">
      <button>Qo'shish</button>
    </form>

    <div class="filters" role="tablist">
      <button data-filter="all" class="active">Barchasi</button>
      <button data-filter="active">Faol</button>
      <button data-filter="done">Bajarilgan</button>
    </div>

    <ul id="todo-list"></ul>

    <footer class="footer">
      <span id="counter"></span>
      <button id="clear-done">Bajarilganlarni tozalash</button>
    </footer>
  </main>
  <script src="app.js"></script>
</body>
</html>
```

```
// ===== Holat (state) =====
const STORAGE_KEY = 'todos-v1';
let todos = load();
let filter = 'all';

function load() {
  try { return JSON.parse(localStorage.getItem(STORAGE_KEY)) ?? []; }
  catch { return []; }
}

function save() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(todos));
}

// ===== Amallar: holatni o'zgartirib, keyin render =====
function addTodo(text) {
  todos.unshift({ id: crypto.randomUUID(), text, done: false, createdAt:
Date.now() });
  update();
}

function toggleTodo(id) {
  todos = todos.map(t => (t.id === id ? { ...t, done: !t.done } : t));
  update();
}

function deleteTodo(id) {
  todos = todos.filter(t => t.id !== id);
  update();
}

function editTodo(id, text) {
  todos = todos.map(t => (t.id === id ? { ...t, text } : t));
  update();
}

function update() { save(); render(); }

// ===== Render: holatdan DOM chizish =====
const list = document.querySelector('#todo-list');
const counter = document.querySelector('#counter');

function render() {
  const visible = todos.filter(t =>
    filter === 'all' ? true : filter === 'done' ? t.done : !t.done
  );
```

```

list.replaceChildren(...visible.map(createItem));

if (visible.length === 0) {
  const empty = document.createElement('li');
  empty.className = 'empty';
  empty.textContent = 'Hozircha vazifa yo\'q 🤖';
  list.append(empty);
}

const left = todos.filter(t => !t.done).length;
counter.textContent = `${left} ta faol vazifa`;

document.querySelectorAll('[data-filter]').forEach(b =>
  b.classList.toggle('active', b.dataset.filter === filter)
);
}

function createItem(todo) {
  const li = document.createElement('li');
  li.dataset.id = todo.id;
  li.classList.toggle('done', todo.done);

  const checkbox = document.createElement('input');
  checkbox.type = 'checkbox';
  checkbox.checked = todo.done;
  checkbox.dataset.action = 'toggle';

  const span = document.createElement('span');
  span.className = 'text';
  span.textContent = todo.text; // ✅ textContent – XSS'dan
himoya
  span.title = 'Tahrirlash uchun ikki marta bosing';

  const del = document.createElement('button');
  del.dataset.action = 'delete';
  del.setAttribute('aria-label', 'O\'chirish');
  del.textContent = 'x';

  li.append(checkbox, span, del);
  return li;
}

// ===== Hodisalar =====
document.querySelector('#todo-form').addEventListener('submit', (e) => {

```

```

    e.preventDefault();
    const input = e.target.elements.text;
    const text = input.value.trim();
    if (!text) return;
    addTodo(text);
    input.value = '';
    input.focus();
  });

  // Delegatsiya: bitta tinglovchi barcha elementlar uchun
  list.addEventListener('click', (e) => {
    const li = e.target.closest('li[data-id]');
    if (!li) return;
    const action = e.target.dataset.action;
    if (action === 'toggle') toggleTodo(li.dataset.id);
    if (action === 'delete') deleteTodo(li.dataset.id);
  });

  // Ikki marta bosib tahrirlash
  list.addEventListener('dblclick', (e) => {
    const span = e.target.closest('.text');
    if (!span) return;
    const li = span.closest('li');
    const input = document.createElement('input');
    input.value = span.textContent;
    input.className = 'edit';
    span.replaceWith(input);
    input.focus();

    const finish = (saveIt) => {
      const value = input.value.trim();
      if (saveIt && value) editTodo(li.dataset.id, value);
      else render();
    };

    input.addEventListener('keydown', (ev) => {
      if (ev.key === 'Enter') finish(true);
      if (ev.key === 'Escape') finish(false);
    });
    input.addEventListener('blur', () => finish(true), { once: true });
  });

  document.querySelector('.filters').addEventListener('click', (e) => {
    const btn = e.target.closest('[data-filter]');
    if (!btn) return;

```



```
filter = btn.dataset.filter;
render();
});

document.querySelector('#clear-done').addEventListener('click', () => {
  todos = todos.filter(t => !t.done);
  update();
});

render();
```

```

body { font-family: system-ui, sans-serif; background: #f1f5f9; margin: 0; }

.app { max-width: 520px; margin: 48px auto; background: #fff; padding: 28px; border-radius: 16px; box-shadow: 0 10px 30px rgb(0 0 0 / .06); }

#todo-form { display: flex; gap: 8px; }
#todo-form input { flex: 1; padding: 12px; border: 1px solid #cbd5e1; border-radius: 10px; font: inherit; }
button { font: inherit; cursor: pointer; }
#todo-form button { padding: 0 18px; border: 0; border-radius: 10px; background: #4f46e5; color: #fff; }

.filters { display: flex; gap: 6px; margin: 16px 0; }
.filters button { border: 1px solid #e2e8f0; background: #fff; padding: 6px 12px; border-radius: 999px; }
.filters button.active { background: #eef2ff; border-color: #4f46e5; color: #4f46e5; }

#todo-list { list-style: none; padding: 0; margin: 0; }
#todo-list li { display: flex; align-items: center; gap: 10px; padding: 12px 4px; border-bottom: 1px solid #f1f5f9; }
#todo-list li .text { flex: 1; }
#todo-list li.done .text { text-decoration: line-through; color: #94a3b8; }
#todo-list li button { border: 0; background: none; color: #ef4444; opacity: .5; }
#todo-list li:hover button { opacity: 1; }

.edit { flex: 1; padding: 6px; font: inherit; }
.empty { justify-content: center; color: #94a3b8; }
.footer { display: flex; justify-content: space-between; margin-top: 16px; color: #64748b; font-size: 14px; }
.footer button { border: 0; background: none; color: #64748b; text-decoration: underline; }

```

MISOL 2 Modal oyna (accessibility bilan)

HTML'ning o'rnatilgan `<dialog>` elementi fokusni boshqarish va `Esc` bilan yopishni o'zi bajaradi.

```

<button id="open-modal">Buyurtmani tasdiqlash</button>

<dialog id="confirm-modal" aria-labelledby="modal-title">
  <form method="dialog">
    <h2 id="modal-title">Buyurtmani tasdiqlaysizmi?</h2>
    <p>Jami: <b>1 105 200 so'm</b>. Buyurtma 2 kun ichida yetkaziladi.</p>
    <menu>
      <button value="cancel">Bekor qilish</button>
      <button value="confirm" class="primary">Tasdiqlash</button>
    </menu>
  </form>
</dialog>

<p id="status"></p>

<style>
  dialog { border: 0; border-radius: 16px; padding: 28px; max-width:
420px; box-shadow: 0 20px 60px rgb(0 0 0 / .25); }
  dialog::backdrop { background: rgb(15 23 42 / .55); backdrop-filter:
blur(3px); }
  dialog[open] { animation: pop .2s ease; }
  @keyframes pop { from { opacity: 0; transform: scale(.95); } }
  menu { display: flex; justify-content: flex-end; gap: 8px; padding: 0; }
  .primary { background: #4f46e5; color: #fff; border: 0; padding: 8px
16px; border-radius: 8px; }
</style>

<script>
  const modal = document.querySelector('#confirm-modal');
  const status = document.querySelector('#status');

  document.querySelector('#open-modal').addEventListener('click', () =>
modal.showModal());

  // Fon (backdrop) bosilganda yopish
  modal.addEventListener('click', (e) => {
    if (e.target === modal) modal.close('cancel');
  });

  // Qaysi tugma bilan yopilganini bilish
  modal.addEventListener('close', () => {
    status.textContent = modal.returnValue === 'confirm'

```

```
    ? '✅ Buyurtma qabul qilindi!'  
    : 'Buyurtma bekor qilindi.';  
  });  
</script>
```

`<form method="dialog">` ichidagi tugma bosilganda dialog avtomatik yopiladi va tugmaning `value` si `modal.returnValue` ga yoziladi. `showModal()` orqa fondagi sahifani "faolsizlantiradi" — klaviatura fokusi oynadan tashqariga chiqmaydi.

MISOL 3 Real vaqtda forma validatsiyasi

HTML

```
<form id="register" novalidate>
  <div class="field">
    <label for="username">Foydalanuvchi nomi</label>
    <input id="username" name="username">
    <small class="error"></small>
  </div>
  <div class="field">
    <label for="email">Email</label>
    <input id="email" name="email" type="email">
    <small class="error"></small>
  </div>
  <div class="field">
    <label for="password">Parol</label>
    <input id="password" name="password" type="password">
    <div class="meter"><span></span></div>
    <small class="error"></small>
  </div>
  <div class="field">
    <label for="confirm">Parolni tasdiqlang</label>
    <input id="confirm" name="confirm" type="password">
    <small class="error"></small>
  </div>
  <button type="submit">Ro'yxatdan o'tish</button>
</form>

<style>
  .field { margin-bottom: 14px; display: grid; gap: 4px; }
  input { padding: 10px; border: 2px solid #cbd5e1; border-radius: 8px; }
  .field.invalid input { border-color: #ef4444; }
  .field.valid input { border-color: #22c55e; }
  .error { color: #ef4444; min-height: 1.2em; }
  .meter { height: 6px; background: #e2e8f0; border-radius: 3px; overflow:
hidden; }
  .meter span { display: block; height: 100%; width: 0; transition: width
.3s, background .3s; }
</style>

<script>
```

```

const form = document.querySelector('#register');

// Har bir maydon uchun qoidalar: xato matnini yoki null qaytaradi
const rules = {
  username: (v) =>
    !v ? 'Majburiy maydon' :
    !/^[a-z0-9_]{3,20}$/i.test(v) ? '3-20 ta lotin harf, raqam yoki _' :
  null,
  email: (v) =>
    !v ? 'Majburiy maydon' :
    !/^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/i.test(v) ? 'Email noto\'g\'ri' :
  null,
  password: (v) =>
    v.length < 8 ? 'Kamida 8 ta belgi' :
    !/\d/.test(v) || !/[A-Za-z]/.test(v) ? 'Harf va raqam bo\'lishi
shart' : null,
  confirm: (v, all) => v !== all.password ? 'Parollar mos emas' : null,
};

function validateField(name) {
  const values = Object.fromEntries(new FormData(form));
  const error = rules[name](values[name] ?? '', values);
  const field = form.elements[name].closest('.field');
  field.querySelector('.error').textContent = error ?? '';
  field.classList.toggle('invalid', !!error);
  field.classList.toggle('valid', !error);
  return !error;
}

function passwordStrength(p) {
  let score = 0;
  if (p.length ≥ 8) score++;
  if (p.length ≥ 12) score++;
  if (/[A-Z]/.test(p) && /[a-z]/.test(p)) score++;
  if (/\\d/.test(p)) score++;
  if (/^[^A-Za-z0-9]/.test(p)) score++;
  return score; // 0..5
}

form.addEventListener('input', (e) => {
  const name = e.target.name;
  validateField(name);
  if (name === 'password') {
    if (form.elements.confirm.value) validateField('confirm');
  }
});

```

```

    const s = passwordStrength(e.target.value);
    const bar = form.querySelector('.meter span');
    bar.style.width = `${s * 20}%`;
    bar.style.background = ['#ef4444', '#ef4444', '#f59e0b', '#eab308',
'#22c55e', '#16a34a'][s];
  }
});

form.addEventListener('submit', (e) => {
  e.preventDefault();
  const ok = Object.keys(rules).map(validateField).every(Boolean);
  if (!ok) {
    form.querySelector('.invalid input').focus();
    return;
  }
  const { confirm, ...data } = Object.fromEntries(new FormData(form));
  console.log('Serverga yuboriladi:', data); // 6-darsda fetch() bilan
yuboramiz
});
</script>

```

Diqqat qiling: barcha maydonlar uchun **bitta** `input` tinglovchisi (delegatsiya); qoidalar ma'lumot sifatida (`rules` obyekti) saqlangan — yangi maydon qo'shish uchun faqat bitta qoida yozish kifoya. `novalidate` esa brauzerning standart xabarlarini o'chirib, o'zimiznikini ko'rsatishga imkon beradi.

HTML

```

<div class="tabs" data-tabs>
  <div role="tablist">
    <button role="tab" aria-selected="true" aria-controls="p1"
id="t1">Tavsif</button>
    <button role="tab" aria-selected="false" aria-controls="p2" id="t2"
tabindex="-1">Xususiyatlar</button>
    <button role="tab" aria-selected="false" aria-controls="p3" id="t3"
tabindex="-1">Sharhlar (12)</button>
  </div>
  <section role="tabpanel" id="p1" aria-labelledby="t1">Mahsulot
tavsifi... </section>
  <section role="tabpanel" id="p2" aria-labelledby="t2" hidden>Texnik
xususiyatlar... </section>
  <section role="tabpanel" id="p3" aria-labelledby="t3" hidden>Xaridorlar
sharhlari... </section>
</div>

<h2>Ko'p beriladigan savollar</h2>
<div class="faq">
  <details><summary>Yetkazib berish qancha vaqt oladi?</summary>
<p>Toshkent bo'ylab 1 kun, viloyatlarga 2-4 kun.</p></details>
  <details><summary>Mahsulotni qaytarish mumkinmi?</summary><p>Ha, 10 kun
ichida, qadoq buzilmagan bo'lsa.</p></details>
  <details><summary>Qanday to'lov usullari bor?</summary><p>Click, Payme,
Uzum, naqd pul.</p></details>
</div>

<script>
  // Tablar: sichqoncha va klaviatura (← →) bilan
  document.querySelectorAll('[data-tabs]').forEach((root) => {
    const tabs = [...root.querySelectorAll('[role="tab"]')];

    function select(tab) {
      tabs.forEach((t) => {
        const active = t ≡ tab;
        t.setAttribute('aria-selected', active);
        t.tabIndex = active ? 0 : -1;
        root.querySelector('#' + t.getAttribute('aria-controls')).hidden =

```



```

!active;
  });
  tab.focus();
}

root.addEventListener('click', (e) => {
  const tab = e.target.closest('[role="tab"]');
  if (tab) select(tab);
});

root.addEventListener('keydown', (e) => {
  const i = tabs.indexOf(document.activeElement);
  if (i === -1) return;
  if (e.key === 'ArrowRight') select(tabs[(i + 1) % tabs.length]);
  if (e.key === 'ArrowLeft') select(tabs[(i - 1 + tabs.length) %
tabs.length]);
});
});

// Akkordeon: bir vaqtda faqat bitta savol ochiq bo'lsin
const faq = document.querySelector('.faq');
faq.addEventListener('toggle', (e) => {
  if (!e.target.open) return;
  faq.querySelectorAll('details[open]').forEach((d) => {
    if (d !== e.target) d.open = false;
  });
}, true); // toggle hodisasi bubbling qilmaydi – capturing bosqichida
ushlaymiz
</script>

```

Bu yerda ARIA atributlari (`role`, `aria-selected`, `aria-controls`) ekran o'quvchilarga komponent tuzilishini tushuntiradi, `tabIndex` esa "roving tabindex" usuli bilan klaviatura navigatsiyasini to'g'ri qiladi. Akkordeonda `toggle` hodisasi ko'tarilmagani uchun `addEventListener` ning uchinchi argumenti `true` (capturing) ishlatilgan.

Mashqlar



Mustaqil ish

1. Todo ilovasiga quyidagilarni qo'shing: muddat (`date`), muhimlik darajasi (rangli belgi), drag & drop bilan tartiblash.
2. Rasm galereyasi: kichik rasmlar bosilganda katta ko'rinishda (lightbox) ochilsin, `←` `→` bilan almashsin, `Esc` bilan yopilsin.
3. "Xarid kalkulyatori": mahsulotlar ro'yxati, har birida +/- tugmalari, jami summa real vaqtda yangilansin (4-dars 1-misol funksiyalaridan foydalaning).
4. Sahifa pastga scroll qilinganda paydo bo'ladigan "↑ Yuqoriga" tugmasini yasang.
5. Kalit so'z bo'yicha jadval qatorlarini filtrlaydigan qidiruv maydonini yarating; mos kelgan qism `<mark>` bilan belgilansin (XSS'dan ehtiyot bo'ling!).



Xulosa

- DOM — HTML'dan qurilgan jonli obyektlar daraxti; `querySelector` / `closest` bilan topiladi.
- Foydalanuvchi ma'lumoti uchun `textContent` ; `innerHTML` — XSS xavfi.
- Uslubni `classList` orqali o'zgartiring; ma'lumotni `data-*` va `dataset` da saqlang.
- Hodisalar bubbling qiladi — delegatsiya bilan bitta tinglovchi ko'plab elementga xizmat qiladi.
- Formalarda `preventDefault()` + `FormData` .
- Holat (state) → render arxitekturasida kodni tartibli qiladi va React'ga tayyorlaydi.

☐ Darsni o'qib bo'ldim